

动漫图形系统综合训练 I

实验指导书 (L1-L14)

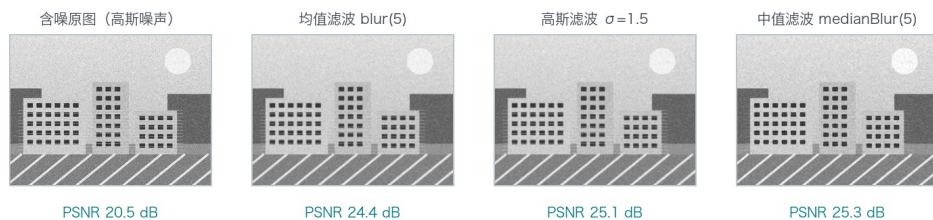
从 OpenCV 环境搭建到增量式三维重建 : 14 次实验 $\times$ 软件工程全过程

配套理论课 : 数字图像基础 (T1-T3) $\rightarrow$ 视觉特征与匹配 (T4-T8) $\rightarrow$ 相机几何与重建
(T9-T10)

实验三 空间滤波与图像增强

软件工程环节 : 系统架构设计 (画 SFM 系统数据流图、标注本实验模块边界) + 接口约定
(统一函数签名与数据结构) | 对应理论课 : T3 灰度变换、直方图与空间滤波

L3 空间滤波效果对比 (高斯噪声 $\sigma=25$)



三者都明显降噪: 高斯滤波最自然, 中值滤波对脉冲噪声更有效, 均值滤波边缘损失最大

图 1 均值/高斯/中值滤波去噪效果对比 (实测结果 , 含 PSNR)

L3 图像锐化（边缘增强）



图 2 拉普拉斯与 USM 锐化效果（实测结果，含平均梯度）

0. 学前准备

- 前置知识：复习 T3 空间滤波章节，理解“卷积核在图像上滑动”的过程。
- 软件准备：pip install scikit-image（本实验 SSIM 评价需要；若安装慢可用清华镜像源）。
- 数据准备：准备一张纹理丰富的照片（如建筑、街道），用于观察滤波对边缘的影响。
- 预习概念：能区分“高斯噪声”（随机灰度波动）与“椒盐噪声”（黑白散点）的外观差异。

1. 实验背景

空间滤波是图像增强的核心工具：卷积核在图像上滑动，以邻域加权和替换中心像素。均值滤波与高斯滤波用于平滑去噪，中值滤波对椒盐噪声尤其有效，拉普拉斯算子和 USM 锐化则用于突出边缘细节。滤波效果直接决定后续特征提取与匹配的质量，是 SFM 流程中“预处理”环节的关键一步。

本实验进入软件工程“设计”阶段的第一环：画出整个 SFM 系统的数据流图（图像→预处理→特征→匹配→几何→点云→可视化），明确本实验（预处理模块）的输入输出边界，并与其他小组约定统一的模块接口（函数签名与数据结构）。

2. 实验目的

1. 理解卷积与空间滤波的原理，掌握均值、高斯、中值滤波的适用场景与参数。
2. 掌握锐化方法（拉普拉斯、USM），理解锐化增强边缘的原理。
3. 会用 PSNR/SSIM 定量评价滤波与增强效果。
4. 绘制 SFM 系统数据流图并定义模块接口约定，完成设计阶段第一份产出。

3. 实验要求

1. 任务 1（滤波对比）：对含高斯噪声的图片分别做均值滤波（`cv2.blur`）、高斯滤波（`cv2.GaussianBlur`）、中值滤波（`cv2.medianBlur`），用同一组参数比较去噪效果与边缘保留情况。

2. 任务 2（椒盐噪声）：为图片添加椒盐噪声（随机置 0 或 255），对比中值滤波与其他滤波在该场景下的效果。

3. 任务 3（锐化）：用拉普拉斯算子（`cv2.Laplacian`）与 USM 锐化（`filter2D` 或自定义）增强图像，观察边缘变清晰的效果。

4. 任务 4（质量评价）：实现 PSNR 计算（公式： $10 \cdot \log_{10}(255^2/\text{MSE})$ ），并调用 `skimage.metrics.structural_similarity` 计算 SSIM（若未安装 `skimage` 则 `pip install scikit-image`），对滤波结果给出定量评分表。

5. 任务 5（软件工程-架构设计）：小组绘制 SFM 系统数据流图（可用 draw.io/Excalidraw/PPT），标注：每个模块的输入/输出、本实验“预处理模块”的边界、数据格式约定（如统一 `uint8 BGR`）。

6. 任务 6（软件工程-接口约定）：与小组内其他同学约定统一函数签名，例如：
`def preprocess(img_bgr: np.ndarray, cfg: dict) -> np.ndarray`，并把约定写入 `docs/interface_agreement.md`。

7. 加分项：实现自适应中值滤波或双边滤波 (cv2.bilateralFilter) ，对比保边去噪效果。

4. 实验原理

4.1 空间滤波原理

空间滤波的本质是卷积：输出图像每个像素是核覆盖邻域内的加权和，

$$g(x, y) = \sum_i \sum_j w(i, j) \cdot f(x + i, y + j)$$

其中 w 为核 (如 3×3 均值核所有权重为 $1/9$) 。均值核平滑但损失边缘；高斯核按二维高斯分布加权 (σ 控制平滑程度) ，更柔和、更保边；中值滤波用邻域中值替换中心像素，对脉冲 (椒盐) 噪声有奇效。

锐化：拉普拉斯算子 $\nabla^2 f$ 提取二阶导数信息 (突出灰度突变处) ，增强图像 $g = f - \nabla^2 f$ ；USM (Unsharp Mask) 先模糊再与原图加权差，

$$g = f + \alpha \cdot ((f - \text{blur}(f)))$$

α 控制增强强度 (本实验取 0.6 ，即 $\text{img} \cdot 1.6 - \text{blur} \cdot 0.6$) 。

质量评价：

$$PSNR = 10 \cdot \log_{10} \left(\frac{255^2}{MSE} \right)$$

PSNR 越大表示与无噪声原图越接近 (通常 > 28 dB 视为去噪有效) ；SSIM 衡量结构相似性，更符合人眼感知，取值越接近 1 越好。

4.2 本实验用到的函数

cv2.blur(src, (ksize, ksize))：均值滤波。

cv2.GaussianBlur(src, (0,0), sigma)：高斯滤波，sigma 控制模糊程度。

`cv2.medianBlur(src, ksize)`：中值滤波，`ksize` 必须为奇数。

`cv2.Laplacian(src, cv2.CV_64F) / cv2.filter2D(src, ddepth, kernel)`：拉普拉斯与自定义核卷积。

PSNR：`psnr = 10*np.log10(255.0**2 / mse)`，`mse` 为均方误差。

5. 实验步骤 (详细)

步骤 1 准备含噪图像：读入图片，用 `numpy` 生成高斯噪声与椒盐噪声各一张，保存到 `results/`。

```
rng = np.random.default_rng(0)
img = cv2.imread("data/photo01.jpg", 0)
gau = np.clip(img + rng.normal(0, 25, img.shape), 0, 255).astype(np.uint8)
salt = img.copy()
mask = rng.random(img.shape)
salt[mask < 0.05] = 0; salt[mask > 0.95] = 255
cv2.imwrite("results/l3_gaussian_noise.jpg", gau)
cv2.imwrite("results/l3_salt_pepper.jpg", salt)
```

两张噪声图生成成功：高斯噪声呈均匀“雪花”，椒盐噪声为黑白散点。

步骤 2 三种滤波对比：对高斯噪声图依次用 `blur(5)`、`GaussianBlur( $\sigma=1.5$ )`、`medianBlur(5)`，并排保存 `1×4` 对比图。

```
b1 = cv2.blur(gau, (5, 5))
b2 = cv2.GaussianBlur(gau, (0, 0), 1.5)
b3 = cv2.medianBlur(gau, 5)
for n, r in [("blur", b1), ("gauss", b2), ("median", b3)]:
    cv2.imwrite(f"results/l3_{n}.jpg", r)
```

高斯滤波对高斯噪声去噪最自然；中值滤波边缘略硬；三者都明显降低噪声。

步骤 3 椒盐噪声场景：对椒盐噪声图执行同一组滤波，特别观察中值滤波的效果。

两级噪声的生成代码与上面的结论逐条对应，便于对照观察：

```
b1s = cv2.blur(salt, (5, 5))
b2s = cv2.GaussianBlur(salt, (0, 0), 1.5)
b3s = cv2.medianBlur(salt, 5)
```

```
for tag, im in [("salt", salt), ("blur", b1s), ("gauss", b2s), ("median", b3s)]:
```

```
    cv2.imwrite(f"results/l3_sp_{tag}.jpg", im)
    print(tag, "PSNR:", round(psnr(img, im), 2))
```

要点：把步骤 2 (高斯噪声) 与步骤 3 (椒盐噪声) 的 PSNR 放在同一张表里，才能看出“不同噪声要配不同滤波器”这一结论。

中值滤波几乎完全去除椒盐点且边缘保持最好；均值/高斯滤波会留下灰斑。

步骤 4 锐化：对原图做拉普拉斯增强与 USM 增强，输出并排对比图。

```
lap = cv2.Laplacian(img, cv2.CV_64F)
sharp_lap = np.clip(img.astype(np.float64) - 0.8 * lap, 0, 255).astype(np.uint8)
blur = cv2.GaussianBlur(img, (0, 0), 2.0)
sharp_usm = cv2.addWeighted(img, 1.6, blur, -0.6, 0)
```

锐化后边缘与纹理更清晰；USM 比直接拉普拉斯增强更自然、噪声放大更小。

步骤 5 质量评价：计算 PSNR 与 SSIM，输出一张评分表（原图/高斯噪声/滤波后三行）。

```
def psnr(a, b):
    mse = np.mean((a.astype(np.float64) - b.astype(np.float64)) ** 2)
    return 10 * np.log10(255.0 ** 2 / mse)
from skimage.metrics import structural_similarity as ssim
print("PSNR blur:", psnr(img, b1), " SSIM:", ssim(img, b1))
```

滤波后 PSNR/SSIM 明显高于含噪图（例如 PSNR 从 ~20dB 提升到 ~28dB 以上），表格记录各方法数值。

步骤 6 软件工程-数据流图与接口：小组完成 SFM 数据流图与接口约定文档；在汇报中说明预处理模块的输入输出（输入：BGR 图像；输出：去噪/增强后的 BGR 图像），并演示一张图走完“读入→预处理”的接口调用。

docs/ 下新增 architecture.md（数据流图）与 interface_agreement.md（接口签名）；图中模块边界清晰。

6. 常见问题与排错

问题：滤波后图像变模糊/边缘丢失 解决：核越大越模糊。均值滤波没有权重，边缘保持最差；需要保边请用高斯或中值，或增大 sigma 而非核尺寸。

问题：椒盐噪声去不掉 解决：均值/高斯对椒盐噪声几乎无效，必须用中值滤波（或自定义邻域中值）。

问题：拉普拉斯锐化出现黑边/过曝 解决：拉普拉斯增强会放大噪声并产生过冲，控制增强系数（如 0.8 而非 1.5），且先做轻度高斯去噪。

问题：SSIM 报错 shape 不匹配 解决：skimage 的 structural_similarity 要求两图同尺寸同 dtype；先用 cv2.resize 对齐尺寸。

7. 课程思政

精益求精与辩证思维：去噪与保边是一对矛盾——核越大噪声越少、细节也越模糊。优秀的工程师正是在这对矛盾中反复权衡参数，正如“两弹一星”元勋们在极端条件下把每一个部件精度调到极致。本实验的 PSNR/SSIM 定量评价，就是让“差不多”变成“可度量”的科学精神。

8. 提交与评分要点

实验报告（滤波对比图、锐化对比、PSNR/SSIM 评分表）、架构图与接口约定文档、源码 zip。